

Stream and Files :-

Introduction to stream :-

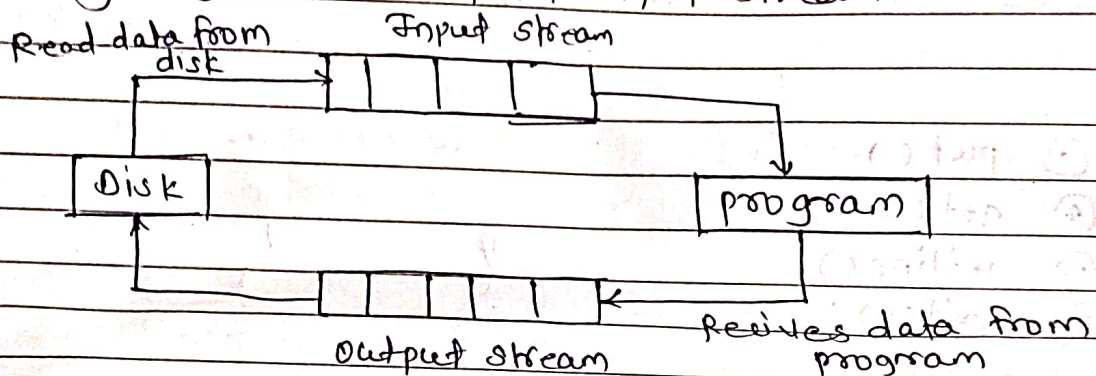
Streams are sequences of bytes used for input/output (I/O) operations. They act as a bridge between the program & input/output devices (e.g. file, keyboards, screens, program).

The stream that supplies data to the program is known as "input stream",

The stream that receives the data from program is known as "output stream".

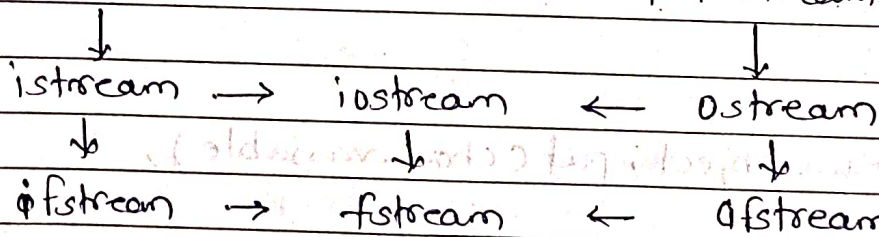
i.e. Input stream reads data from file & output stream write data into file

following diag. shows input/output stream.



In C++ provides following types of stream classes.

ios (input output stream)



ios: is base class for all stream class.

istream: is derived from ios class & used for input operation. e.g. cin, ifstream

ostream: used for output operations e.g. cout, ofstream

iostream: combines istream & ostream for input & output operation (e.g. cin, cout).

ifstream: Drived from istream & used for reading data from file

ofstream: Derived from ostream & used for writing data into files.

fstream: Derived from ifstream & used for both reading & writing the data from file.

* formatted and unformatted I/O functions:-

CH gives us some special functions to read & write characters or raw data directly. These are called unformatted I/O function because they work with raw characters - not formatted like cout, ~~cout~~, cin.

Unformatted I/O function useful when,

- ① if we want to read/write single characters.
- ② if you are working with binary data.

- ① put()
- ② getc()
- ③ getline()
- ④ write()

① put():

It is used when for writing one character to the file.

Syntax:

stream-object.put(char-variable);

② get():

It is used for ~~writing~~ Reading one character from a file.

Syntax

stream-object.get(char-variable);

③ getline():

It is used to read a whole line of text from file.

Syntax:-

getline(stream, string-variable);

(4) Write():

If you writes multiple characters or binary data at time then write() function is used
Syntax of write function is,

stream-object, write(char_array, number_of_characters)

W.A.P to implement get(), put(), getline() & write() function.

```
#include <iostream>
```

```
#include <fstream>
```

```
using namespace std;
```

```
int main()
```

```
{ char ch;
```

```
  ofstream out("output.txt");
```

```
  out.put('H');
```

```
  out.put('i');
```

```
  out.put('i');
```

```
  out.put('\n');
```

```
  out.close();
```

```
  ifstream in("output.txt");
```

```
  while(in.get(ch))
```

```
  {
```

```
    cout << ch;
```

```
  }
```

```
  in.close();
```

```
  ifstream infile("output.txt");
```

```
  string line;
```

```
  while(getline(infile, line))
```

```
  {
```

```
    cout << line << endl;
```

```
  }
```

```
infile.close();
```

```
ofstream out1("binary.bin", ios::binary);
```

```
char data[] = "Hello binary world !";
```

```
out1.write(data, sizeof(data));
```

```
out1.close();
```

```
return 0;
```

```
}
```

* Formatted I/O function:-

Formatted input/output (I/O) are used to reading and writing data in formatted (human readable format) using stream operator (<< &&>>).

C++ provides formatted I/O mainly using,

① cin

② cout

① cin :-

cin is a predefined object of the istream class.

It is used to read input ~~from~~ in formatted using extraction operator (>>). From input devices

Syntax:

```
cin >> input_stream;
```

eg: cin >> rollNo;

② cout :-

cout is a predefined object of the ostream class.

It is used to write output from output devices in formatted (readable format) using insertion operator (<<).

Syntax:

```
cout << output_stream;
```

eg:-

```
cout << Name;
```


Manipulators :-

Manipulators are function that used to control/~~the~~ or change appearance of input or output on the screen or in a file. It is used with input/output streams (cin & cout).

i.e. Manipulators modify how data is displayed or read, like setting decimal places, aligning text, or changing number system.

There are two types of manipulators based on header file

① Basic Manipulator (<iostream>)

② Advanced Manipulator (<iomanip>) input/output manipulators

① Basic Manipulators:- Basic manipulators are defined in iostream. These manipulators don't need any arguments they change how data is displayed directly.

① endl : Add a new line & flushes the output

② hex/oct/dec : changes number format to hexadecimal (base 16), decimal (base 10), or octal (base 8).

③ fixed : shows decimal as-is (e.g. 3.14 instead of 3.14e+00)

④ boolalpha : shows true or false instead of 1 or 0.

eg:-

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int a = 15;
```

```
    float b = 20.55;
```

```
    bool p = true;
```

```
    cout << "Hello" << endl;
```

```
    cout << hex << a << endl;
```

```
    cout << oct << a << endl;
```

```
    cout << boolalpha << p << endl;
```

```
    cout << fixed << b << endl;
```

```
    return 0;
```


Output :

Hello

f

17

true

20.549999

Advanced Manipulators:-

Advanced manipulators are defined in `iomanip` (input/output manipulators). These manipulators takes extra values/arguments to control display how things data.

There are following Advanced manipulators are,

- ① `setw(n)` : It is used to specify the width of the output.
- ② `setprecision(n)` : It is used to set the precision of floating point value.
- ③ `setfill('*')` : fill empty places with * characters
- ④ `setbase(n)` : It is used to set the base value of a numerical number.

c++:-

```
#include <iostream>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int n = 20;
```

```
    double p = 2.50000
```

```
    cout << setw(10) << "Name" << setw(10) << "Age" << endl;
```

```
    cout << setbase(8) << n << endl;
```

```
    cout << fixed << setprecision(4) << p << endl;
```

```
    cout << setw(10) << setfill('*') << "Hi" << endl;
```

```
    return 0;
```

```
}
```

O/P:-

Name Age

24

2.5000

*****Hi

Command line Argument:-

When we run c++ program from the terminal or Dos shell like turbo c++ at that we send additional values (arguments) to the program at the time of program execution.

that values are called as command line arguments. In command line argument we can define `main()` with two arguments, where first argument is the number of command-line arguments & the second is the list of command-line arguments.

Syntax.

```
void main( int argc, char * argv[] )
{
    // Body of main function.
}
```

Here,

`argc` :- is the ^{int} variable that count total number of arguments pass and also ~~count~~ count the name of program.

`argv[]` :- is array of C-style strings (char*) containing the arguments.

Note:-

- ① `argv[0]` always contain the name/path of the program
- ② User arguments are stored from `argv[1]` upto `argv[argc-1]`.
- ③ All the arguments are stored as strings. You must convert them to other types (e.g, int, float)

W.A.P. to implement command-line argument program which shows total number of argument & show them. (Using turbo c++)

```
#include <iostream.h>
#include <conio.h>
void main( int argc, char * argv[] )
{
    int i;
```


Command line Argument:-

When we run c++ program from the terminal or Dos shell like turbo c++ at that we send additional values (arguments) to the program at the time of program execution.

That values are called as command line arguments.

In command line argument we can define `main()` with two arguments, where first argument is the number of command-line arguments & the second is the list of command-line arguments.

Syntax.

```
void main( int argc, char * argv[] )
```

```
{
```

```
// Body of main function.
```

```
}
```

Here,

`argc` :- is the ^{int} variable that count total number of arguments pass and also ~~count~~ count the name of program.

`argv[]` :- is array of C-style strings (`char*`) containing the arguments.

Note:-

- ① `argv[0]` always contain the name/path of the program
- ② User arguments are stored from `argv[1]` upto `argv[argc-1]`.
- ③ All the arguments are stored as strings. You must convert them to other types (e.g, int, float)

W.A.P. to implement command-line argument program which shows total number of argument & show them.
(Using turbo c++)

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
void main( int argc, char * argv[] )
```

```
{ int i;
```



```

    closes(c);
    cout << "Total arguments: " << argc << endl;
    for(i=0; i<argc; i++)
    {
        cout << "Argument [" << i << "] : " << argv[i] << endl;
    }
    getch();
}

```

- ① To run above program first save it with filename.cpp extension.
- ② Then open Dos shell from (File → Dos shell) in turbo c++.
- ③ After that type filename.exe argument on Dos shell & press enter it will gives total number of arguments & argument.

Note:- The atoi() function is used to change/convert a C style string (char*) to an int.

It is defined in <stdlib.h> header in c++.

eg:-